Politecnico di Milano

Advanced Programming for Scientific Computing,
taught by Prof. Luca Formaggia
Numerical Analysis for Partial Differential Equations,
taught by Prof Paola F. Antonietti
Master's Degree in Mathematical Engineering
School of Industrial and Information Engineering

DL-ROM for the electrostatic force in an idealized MEMS

Project by

Alessandro Pedone
Student ID 273814

Marta Pignatelli
Student ID 273530

Supervisors:

Prof. Andrea Manzoni
Dr. Filippo Zacchei

It meant so much to work together
during a special moment in our lives.

Abstract

This project tackles the simulation of coupled electrostatic and mechanical phenomena in MEMS (Micro-Electro-Mechanical Systems) devices, whose widespread adoption in contemporary technologies makes accurate yet efficient modeling essential.

Full-order numerical solvers quickly become computationally prohibitive for realistic geometries, and in such cases one possible solution is to rely on reduced-order modeling.

We therefore adopt this approach: we construct an idealized test case that retains the key physical features, then we develop a reduced-order model based on recent deep-learning-based ROM (DL-ROM) techniques, in particular DeepONets, using an approximation of mechanical deformation grounded in Euler–Bernoulli beam theory with the aim of interfacing it with a finite-element solver for computing the mechanical response.

A brief review frames our methodology within current trends in MEMS modeling and data-driven model reduction, together with classical foundations such as modal decomposition techniques.

Tools: `gmsh` has been used for geometry and meshing; FEniCSx for high-fidelity data generation; TensorFlow/Keras for the network architecture; and Sphinx for the documentation.

Keywords: MEMS, multi-physics, DeepONet, DL-ROM, reduced-order modeling, Euler-Bernoulli beam theory, modal approximation.

Repository: https://github.com/alessandropedone/deeponet-for-mems.

Contents

Chapter 1
MEMS modeling

This chapter motivates the problem by reviewing fundamental concepts in MEMS modeling, introducing a simplified (idealized) MEMS model, and outlining the structure of the present work.

1.1 Introduction to MEMS

Micro-Electro-Mechanical Systems (MEMS) are a technology that we encounter across several devices we use daily. They involve micro-structural and electronic components to realize various functions, such as accelerometers, gyroscopes, magnetometers, scanners and pressure sensors. The challenge implied with their utilization is due to their small scale characteristic, which goes down to the micron order: because of this, the fabrication process is complex and prone to result in discrepancies between the nominal layout expected and the actual one. As reported in the literature (see [MAM18] and [HLK00]), the deviation from the intended design in standard MEMS can reach a *10% error*, which naturally entails modifications to the operational characteristics of the system.
This is why, to ensure correct measurements and avoid compromising the target performance, an *accurate calibration* of each individual device is required. Calibration consists in identifying the deterministic parameters characterizing the specific sensor, such as biases, scale factors, and misalignments.
Due to residual manufacturing variability, measurement noise, and modeling inaccuracies, these parameters cannot be estimated exactly. This motivates formulating the calibration problem through an *Uncertainty Quantification* (UQ) approach, which allows not only the estimation of the calibration parameters, but also the characterization of the associated estimation uncertainty.[1] Within this perspective, the calibration task can be performed:

- by *high-fidelity* or *full-order* models (FOM), which means relying on numerical simulations, and especially Finite Element (FE) solvers, which could solve the problem but would still require infeasible computational resources;
- by *low-fidelity* or *reduced-order (surrogate)* models, which, while being less resource-intensive, still provide an acceptable level of accuracy and expedite repeated computations. Some possible options are: data-fit models, hierarchical models and reduced-order models (ROM).

1.2 Study case

We will deal with a MEMS capacitive accelerometer and we are interested in addressing its response depending on its exact geometric configuration. So, firstly, we describe its physical/geometric features, we then introduce a simplified/idealized case and in the end we present the mathematical model for the physics of the system, which consists of electrostatic and mechanical aspects.

1.2.1 Geometry of a MEMS accelerometer

In Figure 1.1, the standard geometry of a x-axis MEMS accelerometer is depicted and with this in mind we recall some general facts from [ZRG+24].

- The device comprises a movable mass, shown in orange with no pattern, which functions as a rotor and is anchored to the substrate by two supports, depicted in blue with a chequered pattern.
- The device's body is connected to the anchors through springs composed of folded beams, allowing for motion and compliance.
- The mass is maintained at a ground voltage.
- Parallel to the plates of the movable mass, two sets of electrodes are placed, acting as a stator and fixed to the electrode layer of the MEMS. These are denoted as the Left (in red and with a vertical stripes pattern) and Right (in gray and with a squared grid pattern) electrode groups, with an arbitrary voltage V_l and V_r.
- The capacitances of these groups, denoted as C_l and C_r respectively, vary over time in response to the movement of the accelerometer's body.

However, in the literature, it's generally more common to work on a simplified geometry, like the one in Figure 1.2. This is composed of two conductive plates, that we will refer to as electrodes, maintained at some given potential. In particular, the upper electrode is considered to be a deformable elastic membrane, while the lower is modeled as rigid and fixed. Moreover, the gap H in the undeformed configuration between the two is considered negligible with respect to L, and so we should expect the electric field to behave as if the plates had infinite length, with some effects at the corners of the rectangles.
The design of this kind of devices is based on the interaction between electrostatic and elastic forces. Indeed, applying a voltage difference between the two components in this idealized model generates a Coulomb force which induces, together with the external forces[2], displacements of the membrane and thus transforms electrostatic energy into mechanical energy. The theory behind how we model the induced deformation is explored more in detail in the next section (analytical model) and in Chapter 2 (for its approximation). Building on this concept, we decide to rely on the 2D-geometry schemed in Figure 1.3, whose similarities with the previous model are evident from the figure.

Remark. For details on MEMS modeling, see [You11], particularly Chapter 6.

1.2.2 Coupled mathematical model

Hereby, we present the modeling principles and assumptions that are the object of our study. The goal is to describe the electromechanical problem associated to a MEMS device under external stimuli, i.e. external acceleration, and an imposed driving voltage at the lower plate. The system is governed by two sets of equations:

[1] A detailed treatment of UQ methods is outside the scope of this report.
[2] i.e. the acceleration that the device is meant to detect.

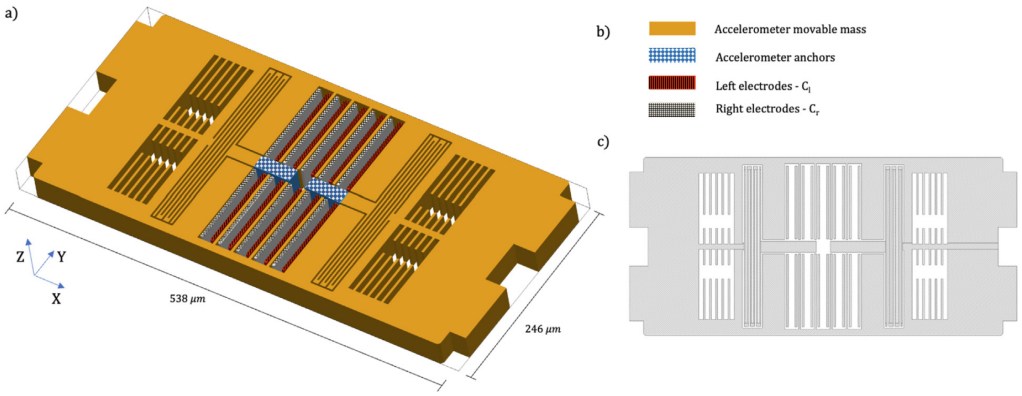


Figure 1.1: x-axis MEMS accelerometer: (a) Device geometry; (b) Color-coded and pattern-coded representation of device regions; (c) Device top view. Source: [ZRG+24].

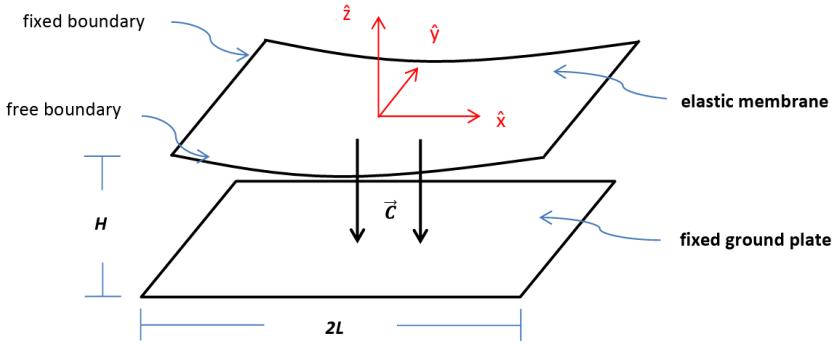


Figure 1.2: Idealized electrostatic MEMS device. Source: [LW16].

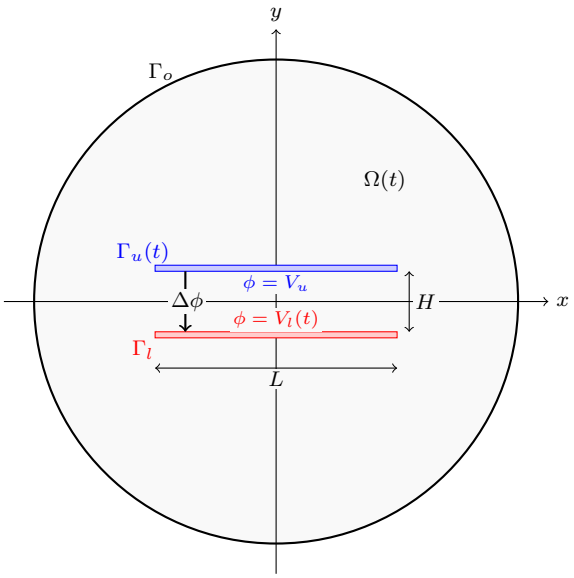


Figure 1.3: Representation of a simplified MEMS. $\Gamma_u(t)$ and Γ_l denote the boundaries of the upper and lower electrodes, respectively. Γ_0 is the outer boundary. V_u and $V_l(t)$ denote the voltages of the upper and lower electrodes, respectively. $\Omega(t)$ denotes the domain enclosed by the electrodes and the outer boundary, i.e. $\partial\Omega(t) = \Gamma_u(t) \cup \Gamma_l \cup \Gamma_o$.

1. one regarding the *mechanical* problem, which aims to get the physical displacement of the upper plate, depending on the different forces they are subject to, through a linear-elastic model;
2. one regarding the *electrostatic* problem, which is meant to provide the electric force, i.e. the source term $\mathbf{t}_u$, for the mechanical problem, by solving for the potential field in the domain, which results from the voltage difference applied to the plates.

Electrostatic part. The problem for the electrostatic potential ϕ is simply the Laplace equation, with Dirichlet boundary conditions on the plates and Neumann or Dirichlet boundary condition[3] for the outer boundary:

$$\begin{cases} -\nabla \cdot (\varepsilon \nabla \phi) = 0 & \text{in } \Omega(t) \\ \phi = V_u & \text{on } \Gamma_u(t) \\ \phi = V_\ell(t) & \text{on } \Gamma_\ell \\ \varepsilon \nabla \phi \cdot n = 0 \quad \text{or} \quad \phi = V_o & \text{on } \Gamma_o \end{cases} \tag{1.1}$$

where $\varepsilon = \varepsilon_0 \varepsilon_r$ is the permittivity and all parameters and domains are defined in the caption of Figure 1.3.

Remark. In the implementation, the following typical driving voltage is used

$$V_\ell(t) = V_{\text{dc}} + V_{\text{ac}} \sin(2\pi f t).$$

Remark. In MEMS modeling, the electric field energy is typically defined as

$$W = \frac{1}{2} \varepsilon \int_\Omega |\nabla \phi|^2 \, dx.$$

In order to relate this field-based quantity to an equivalent lumped parameter description, we introduce a capacitance-like estimate by matching the classical capacitor energy relation

$$W = \frac{1}{2} C (V_l - V_u)^2.$$

This leads to

$$C = \frac{2W}{(V_l - V_u)^2},$$

which defines an effective capacitance associated with the possibly complex and deformable MEMS geometry.

Remark. In this model we treat the domain as a linear dielectric, with constant permittivity coefficient ε_r, and also we assume that quasi-static electrostatics conditions apply, i.e. there are no displacement currents over time.

Mechanical part.[4] We start by stating that our object of interest, the upper plate of the device[5], occupies in the resting/reference configuration the spatial volume Ω_0', and we analyze its motion in a non-inertial reference frame attached to the MEMS system, with material coordinates $\mathbf{X}$. During the time window $t \in \mathcal{T} = (0, t_{\text{final}})$, the object is subject to a deformation characterized by $\mathbf{x} = \chi(\mathbf{X}) = \mathbf{X} + \mathbf{u}(\mathbf{X}, t)$ and we denote the deformed domain by $\Omega'(t)$. The boundary of the reference configuration, $\partial\Omega_0'$, is subdivided into two components: the Dirichlet boundary $\partial\Omega_{0D}'$ and the Neumann boundary $\partial\Omega_{0N}'$. Then, we denote by $\mathbf{N}$ and $\mathbf{n}$ the normal vector entering[6] the plate surface, in the material and current configurations, respectively. The set of equations for the mechanical problem reads as follows:

$$\begin{cases} \rho_0 \ddot{\mathbf{u}}(\mathbf{X}, t; \boldsymbol{\mu}) + \mathbf{C}\dot{\mathbf{u}}(\mathbf{X}, t; \boldsymbol{\mu}) - \nabla_{\mathbf{X}} \cdot \mathbf{P}(\mathbf{u}(\mathbf{X}, t; \boldsymbol{\mu}); \boldsymbol{\mu}) = -\rho_0 \mathbf{a}_0 & \text{in } \Omega_0' \times \mathcal{T}, \quad (1.2) \\ \mathbf{P}(\mathbf{u}(\mathbf{X}, t; \boldsymbol{\mu}); \boldsymbol{\mu}) \cdot \mathbf{N}(\mathbf{X}) = \mathbf{t}_u(\mathbf{X}) & \text{on } \partial\Omega_{0N}' \times \mathcal{T}, \quad (1.3) \\ \mathbf{u}(\mathbf{X}, t) = \mathbf{0} & \text{on } \partial\Omega_{0D}' \times \mathcal{T}, \quad (1.4) \\ \mathbf{u}(\mathbf{X}, 0) = \mathbf{0} & \text{in } \Omega_0', \\ \dot{\mathbf{u}}(\mathbf{X}, 0) = \mathbf{0} & \text{in } \Omega_0', \end{cases}$$

Parameters and physical meaning. The equation (1.2) represents the rate of change of momentum, where ρ_0 represents the initial density in the reference configuration. The vectors $\mathbf{u}, \dot{\mathbf{u}}, \ddot{\mathbf{u}} \in \mathbb{R}^3$ denote displacement, velocity and acceleration of the domain, respectively, and they are the unknowns we solve the equations for. The term $\mathbf{a}_0$ is the acceleration of the non-inertial frame of reference, which corresponds to the acceleration that MEMS aim to detect and represents external forces, as observed from an external viewpoint. We also assume $\mathbf{C}$ and $\mathbf{P}$ to be the damping matrix and the Piola-Kirchhoff stress tensor, respectively[7]. The vector $\boldsymbol{\mu}$, which varies in an hyperectangle of $\mathbb{R}^p$, encodes the set of parameters, such as all the geometrical features that could generate fabrication uncertainties. The last two equations impose the initial conditions of the system, assuming it to be at rest at the beginning of the time window. While, equation (1.3) represents the Neumann boundary condition, with a forcing term given in the material coordinates by:

$$\mathbf{t}_u(\mathbf{x}) = -\frac{1}{2} \varepsilon \left(\frac{\partial \phi(\mathbf{x})}{\partial \mathbf{n}} \right)^2 \mathbf{n} \tag{1.5}$$

that induces the coupling with the electrostatic system. This term corresponds to the electrostatic pressure on the conductor surface and can be seen as a traction $\mathbf{t}_u$ on the upper plate of the device.

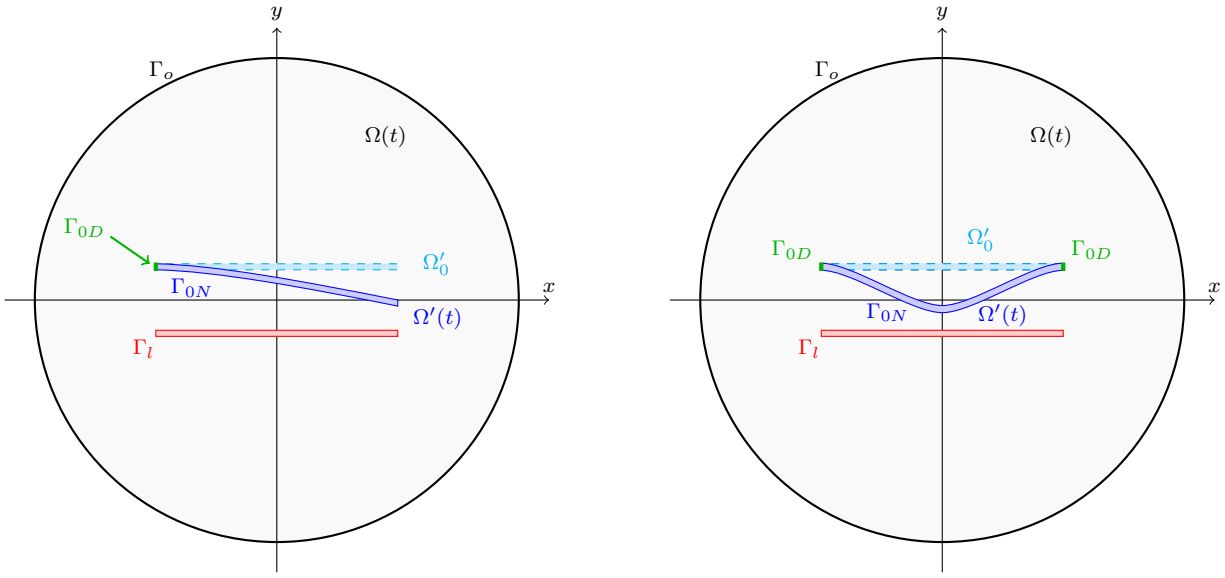
[3] when not specified we assume Neumann boundary conditions.

[4] This part is adapted from [ZRG+24].

[5] Only the upper plate is considered, since the lower plate is fixed.

[6] We make this choice to remain consistent with the quantities defined in the code, where the normal vector is implemented as outer vector of the surface $\Omega(t)$, which is the one being meshed for the electrostatic problem and which is complementary to Ω'.

[7] We won't go deeper defining these matrices, since it is outside the scope of our study.



(a) Cantilever upper electrode.

(b) Clamped–clamped upper electrode.

Figure 1.4: Comparison between a cantilever and a clamped–clamped configuration for the upper electrode. The pinned/clamped boundaries are highlighted and denoted by Γ_{0D}. See [BPS06].

Remark. In a linear dielectric, one can express this quantity equivalently with the formula of the *Maxwell stress tensor*:

$$\mathbf{t}_u = -\mathbf{T}\,\boldsymbol{n}, \quad \mathbf{T} = \varepsilon\left(\mathbf{E}\otimes\mathbf{E} - \frac{1}{2}|\mathbf{E}|^2\mathbf{I}\right)$$

where $\mathbf{E} = -\nabla\phi$ is the electric field and $\mathbf{I}$ is the identity matrix.

Dirichlet boundary conditions. Dirichlet boundary conditions on the plate, represented by equation 1.4, may be of two main kinds, depending on assumptions concerning device-specific characteristics arising from fabrication processes or informed by domain expertise. Here we describe them for our 2D configuration (see Figure 1.4):

a) Cantilever boundary conditions: $\partial\Omega'_D$ contains only the left endpoint of the plate, which is clamped, while the other one is free to move;
b) Clamped-clamped boundary conditions: $\partial\Omega'_D$ contains both of the endpoints of the plate.

1.2.3 Reduced-order model

One could decide to stick with this classical model and implement a FOM based on classical numerical approximations, but, as we have already discussed at the beginning of the chapter, the case of calibration of MEMS could require infeasible computational resources. So, at this point, our goal is to build a reduced-order model. In particular, we want to work on both parts of the coupled physics:

- firstly, leveraging *Euler–Bernoulli beam theory* (see Chapter 2) to simplify the mechanical model and make it much more affordable in terms of computational power, which is something commercial solvers already do;
- then, building a neural network, based on the *DeepONet architecture* (see Chapter 3) and trained on static snapshots, to predict directly the force acting on the upper electrode from the current configuration/displacement of the upper electrode, in order to skip the FEM computation of ϕ, which constitutes the main scientific contribution.

We will call the resulting reduced-order model just *DL-ROM* [FDM21], even if the mechanical component remains a classical modal ROM. Now, we'll discuss more in detail the theoretical framework of our approach.

Chapter 2
Euler-Bernoulli beam theory and ROM

Euler–Bernoulli beam theory, also known as engineer's beam theory or classical beam theory, is a simplification of the linear theory of elasticity, which provides a means of calculating the load-carrying and deflection characteristics of beams. This chapter is devoted to the discussion of the application of this theory in reduced-order modeling and the introduction of our proposed *DL-ROM*.

2.1 Introduction to the Euler-Bernoulli beam

Definition. A *Euler-Bernoulli beam* is characterized by the following properties [You11]:

- *Thin:* its length is much greater than its cross-sectional dimensions, so bending effects dominate.
- *Prismatic:* the cross-section remains constant along the length of the beam.
- *Homogeneous:* the material properties are uniform throughout the beam.
- *Isotropic:* the material properties are the same in all directions.
- *Linear elastic*: the material obeys Hooke's law, meaning stress is proportional to strain. (This assumption is valid since we assume small deformations, while large ones would require a nonlinear theory.)
- *Bernoulli's hypothesis:* which means cross-sections of the beam remain plane, unwarped and perpendicular to the beam axis during bending; this means that the beam can be represented by a shear-rigid mathematical model (see Fig. 2.1).

Notation. We denote the longitudinal axis of the beam with x-axis, while the transverse directions, lying in the plane of the cross-section, are denoted by y and z. In particular, bending occurs in the xy-plane, while z denotes the out-of-plane direction.

Assumptions. For Euler-Bernoulli beams, we have to make the following assumptions to derive the dynamic equations.

Hp	The beam is straight.
	There is no elongation along the x-axis.
	There is no torsion along the x-axis.
	The deformations happen in a single plane.
	The deformations are small.
	The beam has a simple cross section.

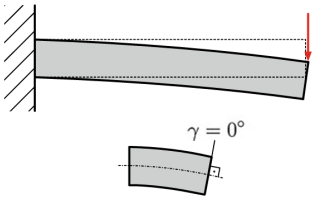


Figure 2.1: Shear-rigid

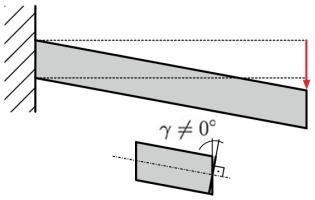


Figure 2.2: Shear-flexible

Motivation. This model can be reasonably applied to describe the deformation of the upper electrode of the idealized MEMS device we introduced before. We can consider the Bernoulli's assumption as *satisfied*, since for homogeneous beams with length about 20 times larger than a characteristic dimension of the cross section, the shear contribution is usually disregarded in the first approximation, even though we model it in our geometries as if it was shear-flexible, for simplicity.

2.2 Euler-Bernoulli beam equations

Free vibration equation. We start by modeling the dynamics of a beam vibrating without any additional forces acting on it. Skipping the mathematical physics details, we can write the following energies for an elastic beam:

$$\text{kinetic energy:} \quad \frac{1}{2}\int_0^L \mu \left(\frac{\partial w}{\partial t}\right)^2 dx$$

$$\text{bending strain energy:} \quad \frac{1}{2}\int_0^L EI \left(\frac{\partial^2 w}{\partial x^2}\right)^2 dx$$

where

- $w = w(x,t)$ describes the deflection, or *displacement*, of the beam in the y direction at some position x at some time t, since we can model the beam as a one-dimensional object thanks to the assumptions;
- E is the elastic modulus, in particular *Young's modulus*, which is defined as the ratio of tensile stress to tensile strain, representing the tendency of the object to deform along an axis when opposing forces are applied along that axis;
- I is the *second moment of area* of the cross section with respect to the bending axis z;
- if we assume ρ to be the material density, A the coss section area, we set $\mu = \rho A$, which represents the *linear mass density* and is assumed to be constant;
- L is the *length* of the beam.

With this in mind, we immediately deduce that the corresponding Lagrangian is

$$\mathcal{L}(w,t) = \int_0^L \left[\frac{1}{2}\mu \left(\frac{\partial w}{\partial t}\right)^2 - \frac{1}{2}EI\left(\frac{\partial^2 w}{\partial x^2}\right)^2\right] dx$$

and that the action is given by

$$I(w) = \int_{t_1}^{t_2}\int_0^L \left[\frac{1}{2}\mu \left(\frac{\partial w}{\partial t}\right)^2 - \frac{1}{2}EI\left(\frac{\partial^2 w}{\partial x^2}\right)^2\right] dx\, dt.$$

After computing the first variations of the action, one finds that the function that minimizes the action solves the following Euler-Lagrange equation:

$$\frac{\partial^2}{\partial x^2}\left(EI\frac{\partial^2 w}{\partial x^2}\right) = -\mu\frac{\partial^2 w}{\partial t^2}.$$

In the case of a homogeneous beam, the product EI, known as the *flexural rigidity*, is constant in space and can consequently be taken out of the derivation sign to get the *free vibration equation*:

$$\boxed{EI\frac{\partial^4 w}{\partial x^4} = -\mu\frac{\partial^2 w}{\partial t^2}.} \tag{2.1}$$

Additional forces. Now that we have a basic model, we want to build on it and add two terms: one responsible for viscous damping and the other modeling a load acting along the y direction. This idea leads to the formulation of the equations of a *damped* Euler-Bernoulli beam:

$$\boxed{\mu\frac{\partial^2 w}{\partial t^2} + c\frac{\partial w}{\partial t} + EI\frac{\partial^4 w}{\partial x^4} = f} \tag{2.2}$$

where $f = f(x,t)$ represents the load and c is the *viscous damping coefficient*. This is the model we want use for our mechanical problem, since it's commonly used to model MEMS [You11].

2.3 Modes of the Euler-Bernoulli beam

Now we focus our attention only on the free vibration equation in (2.1). Qualitatively speaking, we want to find an orthogonal basis of functions of space, which we will call modes and denote by $v_n(x)$, in order to represent the solution w with separated variables as a series

$$w(x,t) = \sum_{n=1}^{\infty} q_n(t)\, v_n(x)$$

From the approximation point of view, this will allow us to do two things:

1. first, we can choose a number of modes N_m to keep and *truncate the series* in this way

$$\boxed{w(x,t) = \sum_{n=1}^{N_m} q_n(t)\, v_n(x);}$$

2. then, if we assume that the functions $v_n(x)$ are known, and indeed they are, we can reduce our problem to a *system of ODEs* in time, with q_n for $n = 1,\dots,N_m$ being the unknowns.

This is important to keep in mind because, in the next section, we will apply the same idea to the general damped equation and propose a numerical approximation scheme for the resulting system of ODEs.

2.3.1 Solution of the free vibration equation (sketch).

Such equation can be solved using separation of variables [KS19]. Indeed, we look for solutions of the form

$$w(x,t) = q(t)\,\hat{w}(x).$$

Plugging this into the equation we get

$$\frac{1}{q}\frac{d^2 q}{dt^2} = -\frac{EI}{\mu}\frac{1}{\hat{w}}\frac{d^4\hat{w}}{dx^4} = -\omega^2$$

Thus, for any frequency of vibration ω, q is a *harmonic oscillator* since it satisfies

$$\ddot{q} + \omega^2 q = 0 \tag{2.3}$$

where we are denoting the derivatives with respect to time using the dots, while, if we denote the with $'$ the derivatives in space, the spatial coefficient $\hat{w}$ is a solution of

$$EI\hat{w}'''' = \mu\omega^2\hat{w}, \tag{2.4}$$

which basically is the *eigenvalue problem* for ∂_x^4. At this point, one could impose a set of *boundary conditions* at $x = 0$ and $x = L$, which describe how the beam's kinematics are constrained at its ends, and apply directly spectral theory to the resulting solution operator to get the existence of a sequence of eigenvalues and of a corresponding orthogonal basis of eigenfunctions. The limit of this approach is that it doesn't provide the explicit form, but actually we can find it by hand. Here we do a sketch of how you proceed to prove the needed properties and to exploit them. First, we notice that the equation (2.4) yields the following general solution:

$$\hat{w}(x) = A_1\cosh(\beta x) + A_2\sinh(\beta x) + A_3\cos(\beta x) + A_4\sin(\beta x) \;\text{ for some }\; A_1,A_2,A_3,A_4 \in \mathbb{R}, \;\text{ with }\; \beta := \left(\frac{\mu\omega^2}{EI}\right)^{1/4}.$$

The constants can be found setting the boundary conditions and for each standard choice, after some computations, one can see that each solution takes the form

$$\hat{w}(x) = Cv(x),$$

where C is an unknown constant (the equation (2.4) is homogeneous!) and v is completely known and depends on ω.
Even if $\hat{w}$ depends on the frequency ω, one can show quite easily, that we cannot put every ω we want: in fact, again from the boundary conditions, we get a constraint that allows us to find a countable set of frequencies, called *natural*

frequencies[1] ω_n, sorted in ascending order, at which the beam can vibrate. Now we define the *modes* $\hat{w}_n$ to be the solution to the problem defined by (2.3.1) with boundary conditions when $\omega = \omega_n$. Obviously, it has to be of the form $\hat{w}_n = C_n v_n$. With some simple integration by parts, it's possible to prove that $\{\hat{w}_n\}$ is the orthogonal basis of eigenfunctions we were looking for, and the same property holds for $\{v_n\}$. So, if we define q_n to be a solution of (2.3) with some suitable initial conditions $q_n(0)$ and $q_n'(0)$ when $\omega = \omega_n$, we immediately get that

$$\{q_n(t)\, v_n(x)\}_{n=1}^{\infty}$$

is a family of solutions to the free vibration problem. Now, we exploit the linearity of the problem to take the series of the family of solutions we found above and get the final displacement w:

$$w(x,t) = \sum_{n=1}^{\infty} q_n(t)\, v_n(x).$$

The initial conditions on $q_n(0)$ can be derived by imposing the initial condition g (which we assume to be regular enough) for w:

$$\sum_{n=1}^{\infty} g_n\, v_n(x) = g(x) = w(x,0) = \sum_{n=1}^{\infty} q_n(0)\, v_n(x),$$

which implies $q_n(0) = g_n$ for every n. The same reasoning applies to $q_n'(0)$.

2.3.2 Boundary conditions

Mode shapes expressions for the most common Euler–Bernoulli boundary conditions are reported in [KS19]. We now discuss two standard choices of boundary conditions: the cantilever beam conditions (like in Figure 1.4a) and clamped-clamped conditions (like in Figure 1.4b). We also want to take a look at the first $N_m = 4$ corresponding modes, since this is the choice we made for the implementation.

Cantilever beam. A cantilever is a structural element that is firmly attached to a fixed structure at one end and it's unsupported at the other end. The free end is supposed not be subject to bending moment, nor shear. Hence, the corresponding boundary conditions are:

$$\boxed{x = 0} \rightarrow\ w(0) = 0,\ w'(0) = 0; \qquad \boxed{x = L} \rightarrow\ w''(L) = 0,\ w'''(L) = 0.$$

For the constant coefficients to be non trivial, we obtain the following relation for the natural frequencies:

$$\cosh(\beta_n L)\cos(\beta_n L) + 1 = 0.$$

By numerically solving the equation in β_n, we then can (approximately) find the natural frequencies $\omega_n = \beta_n^2\sqrt{\frac{EI}{\mu}}$ and the expression of the modes:

$$v_n(x) = \cosh\beta_n x - \cos\beta_n x + \frac{\cos\beta_n L + \cosh\beta_n L}{\sin\beta_n L + \sinh\beta_n L}\,(\sin\beta_n x - \sinh\beta_n x).$$

We plot the first 4 in Figure 2.1.

Clamped-clamped beam. In this case, both vertical displacement and rotation are restrained, so only bending is allowed. This translates into the symmetric conditions:

$$\boxed{x = 0} \rightarrow\ w(0) = 0,\ w'(0) = 0; \qquad \boxed{x = L} \rightarrow\ w(L) = 0,\ w'(L) = 0.$$

For the constant coefficients to be non trivial, we obtain the following relation for the natural frequencies:

$$\cosh(\beta_n L)\cos(\beta_n L) - 1 = 0.$$

By numerically solving the equation in β_n, we then can (approximately) find the natural frequencies $\omega_n = \beta_n^2\sqrt{\frac{EI}{\mu}}$ and the expression of the modes shapes:

$$v_n(x) = \sinh\beta_n x - \sin\beta_n x + \frac{\cos\beta_n L - \cosh\beta_n L}{\sin\beta_n L + \sinh\beta_n L}(\cosh\beta_n x - \cos\beta_n x).$$

We plot the first 4 in Figure 2.1.

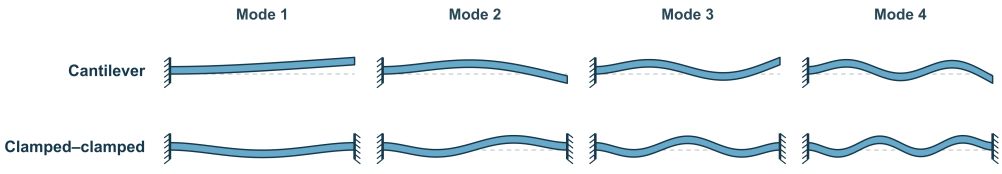


Figure 2.1: First 4 mode shapes $v_{1,2,3,4}$.

[1] Obviously corresponding, up to some multiplicative constant, to the eigenvalues of the solution operator for the problem of $\hat{w}$.

2.4 Numerical approximation of the damped equation

Starting from what we've just discussed, we want to present the numerical strategy we chose to approximate the problem associated to the equation (2.2).

Modal mechanical model. First, we start by applying the ideas of the previous paragraph to the equation (2.2) [You11]. In particular, we look for solutions of the form

$$w(x,t) = \sum_{n=1}^{\infty} q_n(t)\, v_n(x)$$

where v_n are the modes we computed before from the free vibration problem. Now, we plug this expression into the equation and we use the fact that v_n are eigenfunctions of the operator ∂_x^4 to write

$$\mu \frac{\partial^2 w}{\partial t^2} + c\frac{\partial w}{\partial t} + EI\frac{\partial^4 w}{\partial x^4} = \sum_{n=1}^{\infty} (\mu \ddot{q}_n + c\dot{q}_n)\, v_n + EI\, q_n\, v_n'''' = \sum_{n=1}^{\infty} (\mu \ddot{q}_n + c\dot{q}_n + \mu\omega_n^2\, q_n)\, v_n$$

If the force f is regular enough we can write its expansion with respect to the basis $\{v_n\}$, and get some coefficients f_n, which depend on time. So, the equation is satisfied if and only if the coefficients of the two series correspond, i.e. we must have

$$\mu \ddot{q}_n + c\dot{q}_n + \mu\omega_n^2\, q_n = f_n$$

which is just a diagonal[2] infinite dimensional system of ODEs. Thanks to this new modal representation, we can integrate in $[0, L]$, multiply each equation by v_n^2 and write

$$\boxed{m_n \ddot{q}_n + c_n \dot{q}_n + k_n q_n = F_n,} \tag{2.5}$$

where:

- $m_n = \mu \int_0^L v_n^2$ is the modal mass [kg],
- c_n is the modal damping [kg/s],
- $k_n = m_n \omega_n^2$ is the modal stiffness [N/m],
- $F_n = f_n \int_0^L v_n^2$ is the generalized force [N].

This is going to be the system we are interested in solving numerically. It is standard to express c_n in terms of damping ratios ζ_n in this way

$$c_n = 2\zeta_n \omega_n m_n,$$

and therefore we adopt this representation.

Modal approximation. At this point, we make the choice of parametrizing the moving electrode boundary by the first $N_m = 4$[3] mode shapes $\{v_n\}_{n=1}^4$; to be precise we have that

$$w(x,t) \approx \sum_{n=1}^{4} q_n(t)\, v_n(x).$$

There's still one last open question about our theoretical model: how do we compute the forces? And the answer is that we can compute them by the principle of virtual work [You11]. In fact, if we recall that, thanks to our assumptions, the modal shape vector is transverse-only, i.e.

$$\hat{\boldsymbol{v}}_n(x) = \begin{bmatrix} 0 \\ v_n(x) \end{bmatrix},$$

and the expression of the traction in (1.5), we can write

$$\boxed{F_n(t) = b \int_{\Gamma_u(t)} \mathbf{t}_u(s,t) \cdot \hat{\boldsymbol{v}}_n(s)\, ds = b \int_{\Gamma_u(t)} -\frac{1}{2}\varepsilon \left(\frac{\partial \phi}{\partial \boldsymbol{n}}(s)\right)^2 n_2(s)\, v_n(s)\, ds,}$$

where b is the out-of-plane thickness and n_2 denotes the second component of the outward normal with respect to the domain $\Omega(t)$. To compute this force, we can introduce another approximation: if we let $\tilde{\Gamma}_u(t) \subseteq \Gamma_u(t)$ denote the lower edge, we can approximate each modal force by

$$F_n(t) = b \int_{\Gamma_u(t)} \mathbf{t}_u(s,t) \cdot \hat{\boldsymbol{v}}_n(s)\, ds \approx b \int_{\tilde{\Gamma}_u(t)} \mathbf{t}_u(s,t) \cdot \hat{\boldsymbol{v}}_n(s)\, ds,$$

since the evidence tells us that the traction, or equivalently the magnitude of the electric field, is negligible on the other parts of the boundary.

Time integration. For the time integration of (2.5), a Newmark scheme [New59] with $\beta = \frac{1}{4}$, $\gamma = \frac{1}{2}$ is applied independently to each one of the first 4 equations. Given $q^k, \dot{q}^k, \ddot{q}^k$, we define the predictors as:

$$q_{\text{pred}} = q^k + \Delta t\, \dot{q}^k + \frac{\Delta t^2}{2}(1 - 2\beta)\ddot{q}^k$$

$$\dot{q}_{\text{pred}} = \dot{q}^k + \Delta t(1 - \gamma)\ddot{q}^k$$

[2] It means that the ODEs are independent.

[3] See Chapter 5 for more details on this choice.

Then we solve for $\ddot{q}^{k+1}$:

$$\ddot{q}^{k+1} = \frac{F^{k+1} - c\,\dot{q}_{\text{pred}} - k\,q_{\text{pred}}}{m + \gamma\Delta t\,c + \beta\Delta t^2\,k}.$$

Finally, we correct:

$$q^{k+1} = q_{\text{pred}} + \beta\Delta t^2\ddot{q}^{k+1} \qquad \dot{q}^{k+1} = \dot{q}_{\text{pred}} + \gamma\Delta t\ddot{q}^{k+1}$$

In the coupled implementation, F^{k+1} is approximated from the mesh generated by the current modal coordinates (explicit/partitioned coupling). More accurate schemes would use a fixed-point iteration per time step, but this has been deliberately avoided.

Coupling algorithm. Having introduced all the necessary tools, we now outline the coupling algorithm. For time steps $k = 0, \dots, N-1$, we proceed as follows:

1. *Geometry update:* construct the moving electrode boundary from $\{q_n^k\}_{n=1}^4$ and generate the corresponding `.geo` file.
2. *Remeshing:* run `gmsh` to obtain the mesh of the air domain $\Omega(t_k)$.
3. *Electrostatic solve:* compute the discrete potential ϕ_h^k.
4. *Maxwell stress computation:* evaluate the traction $\mathbf{t}_c^k$ on $\tilde{\Gamma}(t_k)$.
5. *Modal projection:* compute the generalized forces $\{F_n^k\}_{n=1}^4$.
6. *Time integration:* update q_n^{k+1}, $\dot{q}_n^{k+1}$, and $\ddot{q}_n^{k+1}$ using the Newmark scheme, for $n = 1, \dots, 4$.
7. *Output:* store ϕ_h^k in a ParaView time series, together with the modal histories (evolution over time of q_n, $\dot{q}_n$ and $\ddot{q}_n$ and execution times.

Classical ROM vs DL-ROM. Up to now, we've discussed the reduced-order modeling of the mechanical part and built a solver, which is a ROM, while relying completely on the classical theory of Euler-Bernoulli beams. We will call this solver *classical ROM*. Now, as already anticipated, we want to exploit a neural network to improve this model. We observe that a big bottleneck of solving the problem with this approach is constructing a new mesh at each time step to compute the electrostatic potential in the whole domain using $\mathbb{P}^1$ elements. This motivates the idea of skipping completely the meshing phase by introducing a neural network that predicts directly the values of the normal derivative of ϕ on $\tilde{\Gamma}$, depending on the current state of system, i.e. its geometry, which is parametrized by the first four modal coefficients q_n. In particular, the neural network we propose takes as input the geometric parameters: q_n, distance between the electrodes H and over-etch of the upper electrode O[4]; and produces as output

$$\frac{\partial\phi}{\partial\boldsymbol{n}}$$

on $\tilde{\Gamma}$. Moreover, we call the resulting coupled solver *DL-ROM*.

Remark. DL-ROM uses *both* reduced-order modeling approaches.

Remark. An important detail to note is that we can avoid to add V_u and V_l (sinusoidal in time) as input parameters to the network. In fact, if you set $V_u = 0$ and $V_l = 1$ and train the network, you can compute the normal derivative for a generic couple of voltages using the following simple transformation[5]:

$$\frac{\partial\phi}{\partial\boldsymbol{n}} = (V_l - V_u)\,\mathcal{D}(\boldsymbol{\mu})$$

where D is the network and $\boldsymbol{\mu} = (O, H, q_1, q_2, q_3, q_4)$ is the vector containing the geometric parameters. Note that the output of the network is actually a *function*.

Remark. We want to keep in mind also that if we want to compute C, our quantity of interest (see Section 1.2.2), we need to compute

$$W = \frac{1}{2}\varepsilon\int_\Omega |\nabla\phi|^2\,dx$$

first. This is possible only if we know ϕ and we are able to integrate it in the whole domain (that means we need a mesh!). To overcome this problem we can rely on a very nice formula to approximate C, namely

$$C(t) \approx \int_0^L \frac{\varepsilon}{H + w(x,t)}\,dx \approx \int_0^L \frac{\varepsilon}{H + \sum_{n=1}^4 q_n(t)\,v_n(x)}\,dx.$$

This allows to compute a good estimation, even if it's quite clear that it usually underestimates (or better it neglects) the *fringing effects*, which for small deformation consist basically of a simple vertical translation of the profile of C over time. A quick fix to this problem is to create the mesh at the first time step and calculate the fringing effects as the difference between the actual capacitance computed using FEM integration and the approximate capacitance. If the deformations are big, the fringing effects may change over time and a translation at the first time step may not be enough, since in any case the profile will slightly, but significantly, differ. In this kind of situations, the solution we propose is to compute the precise capacitance every k^* steps and update the value of the fringing effects, in order to still have an *affordable estimation of the capacitance*.

[4] A scalar parameter that models how much extra material is unintentionally removed beyond the nominal etch depth or boundary during fabrication. In the case of our simplified geometry, it corresponds just to a shrinkage parameter. The precise definition is very intuitive, but it depends on which boundary condition for the beam you choose: for more details, please refer to the `geometry` folder inside the repository, where you can find the `.geo` files containing the precise definition.

[5] The same formula holds if we build a network to predict ϕ instead of its derivative.

Chapter 3
DeepONet architecture

In this chapter we discuss the details concerning the architecture of the neural network used to predict the normal derivative of the electrostatic potential on boundary the upper electrode.

3.1 DeepONet literature summary

Introduction. The DeepONet architecture, proposed by Lu et al. in [LJK19], is part of the framework of *operator learning*. In fact, it was introduced to overcome a key limitation of classical neural networks: they approximate functions, whereas many real-world problems require approximating operators that map an input of various types to a function. The idea behind operator learning is to consider a generic operator between function spaces

$$u \mapsto v = \mathcal{G}(u),$$

and to train a neural network to learn this operator. In other words, you want to get the operator $\mathcal{D}$, represented by a network, such that

$$\mathcal{D}(u) \approx \mathcal{G}(u),$$

in some sense, for every function u in an appropriate space. In our case, we are interested in applying this architecture to a parametric PDE problem. In such scenarios, the input to the operator could be a set of functions and coefficients representing, for example, boundary conditions, a forcing term, geometric or physical characteristics of the problem, etc.

Architecture. This architecture consists of two main subnetworks: the branch network and the trunk network. The outputs of the two networks are vectors of dimension r, which are combined via a *dot product* to produce the final output.

- *Branch network.* The branch network receives as input a discrete representation of the function u, which in our case is simply a vector of geometric parameters. Its role is to extract a latent encoding that captures the global features of the functional input. We denote the output vector of the branch network with $\mathbf{b}(u)$.
- *Trunk network.* The trunk network receives the point $\mathbf{x}$ where we want to evaluate the operator output, i.e. $v = \mathcal{G}(u)$. It produces a basis of functions dependent on the location. We denote the output vector of the trunk network with $\mathbf{t}(\mathbf{x})$.

Consequently, the overall output of the DeepONet is given by:

$$\mathcal{D}(u)(\mathbf{x}) = \sum_{k=1}^{r} b_k(u)\, t_k(\mathbf{x}),$$

where $b_k(u)$ and $t_k(\mathbf{x})$ are elements of the vectors $\mathbf{b}(u)$ and $\mathbf{t}(\mathbf{x})$, respectively.
This structure is motivated by a theorem on operator approximation (see [LJP+21]) which relies on the fact that this decomposition form is inspired by the representation of operators through functional bases, such as Fourier series or eigenfunction expansions.

Training. The loss function corresponds to the sense in which we want to interpret $\mathcal{D}(u) \approx \mathcal{G}(u)$, so by choosing it, we are fundamentally encoding how we want to weight the error between the network output and the desired output. If we assume that the operators $\mathcal{G}$ and $\mathcal{D}$ are maps into a square-integrable function space, which is reasonable if we are solving PDEs, a natural choice for the loss is

$$L(u) = \|\mathcal{D}(u) - \mathcal{G}(u)\|_{L^2}^2.$$

In practice, during training, this loss is approximated by computing the mean squared error (MSE) between the network output and the target values evaluated at various points in the domain:

$$\hat{L}(u) = \frac{1}{N} \sum_{j=1}^{N} |\mathcal{D}(u)(\mathbf{x}_j) - \mathcal{G}(u)(\mathbf{x}_j)|^2 ,$$

where $\{x_j\}_{j=1}^N$ are the points at which the output function is evaluated, and that can be thought of as points of a computational mesh.
In the end, for the training process we assume to have at our disposal N_s solution snapshots, with a *varying number of points* in each snapshot:

$$\left\{ \{\mathcal{G}(u_i)(\mathbf{x}_j)\}_{j=1}^{N_i} \right\}_{i=1}^{N_s} .$$

So the objective function of our minimization will be:

$$\hat{L}(u) = \frac{1}{N_s} \sum_{i=1}^{N_s} \frac{1}{N_i} \sum_{j=1}^{N_i} |\mathcal{D}(u_i)(\mathbf{x}_j) - \mathcal{G}(u_i)(\mathbf{x}_j)|^2 .$$

Remark. The fact that the number of points between different snapshots can change is relevant, since in our application we have a varying domain, and consequently a varying number of degrees of freedom between meshes. The capability of the network to deal with this during training is a crucial problem, since, for statistical reasons, we want to always keep the points of a single combination of parameters together. This means, more practically, that we will split the data in training, validation and test sets and divide the training data into batches only with respect to the snapshot number. To do so, the network must be able to deal with tensor inputs and masked losses, as we will see in the next section.

3.2 Proposed architecture

Going back to our case, we recall that we consider the geometry of the problem as the only variable determining the solution, since changing the voltages corresponds to multiplying by $V_l - V_u$ the output in the case $V_u = 0$ and $V_l = 1$ (see Section 2.4). Hence, the input to the operator will consist solely of the geometric parameters describing the domain shape. The quantities of interest, on the other hand, are actually two. Therefore, our aim is learning two different operators as in Table 3.1. So, inspired by the literature (see [HKA+24]), we use a DeepONet, summarized in the Keras model summary present in tables A.1, A.2, and A.3. Here we give a graphical schematic representation in Figure 3.3.

Operator	Physical meaning
$\boldsymbol{\mu} \mapsto \phi_{\boldsymbol{\mu}}$	electrostatic potential in the entire domain
$\boldsymbol{\mu} \mapsto \frac{\partial \phi_{\boldsymbol{\mu}}}{\partial n}$	normal derivative of the potential boundary of the upper plate (= force up to a constant)

Table 3.1: Operators we want to learn. The vector of parameters $\boldsymbol{\mu} = (O, H, q_1, q_2, q_3, q_4)$ varies in a hyperrectangle in $\mathbb{R}^p$, and the outputs are both functions from $\mathbb{R}^d$ to $\mathbb{R}$.

Remark. We used ReLU as the activation function in the hidden layers with He initialization for the weights, L2 regularization with a parameter of $\lambda = 1 \times 10^{-4}$, dropout with a rate of 0.5, learning rate warm-up, and early stopping.

Remark. The DeepONet includes the hyperparameter `rescale_output`, in addition to the trunk and branch networks. This parameter, which is just a real number, introduces a global scaling of the model output, regulating its magnitude during training. Such rescaling can improve numerical stability and optimization performance by maintaining outputs within a well-conditioned range, thereby alleviating issues related to exploding or vanishing gradients. It is therefore treated as a tunable hyperparameter.

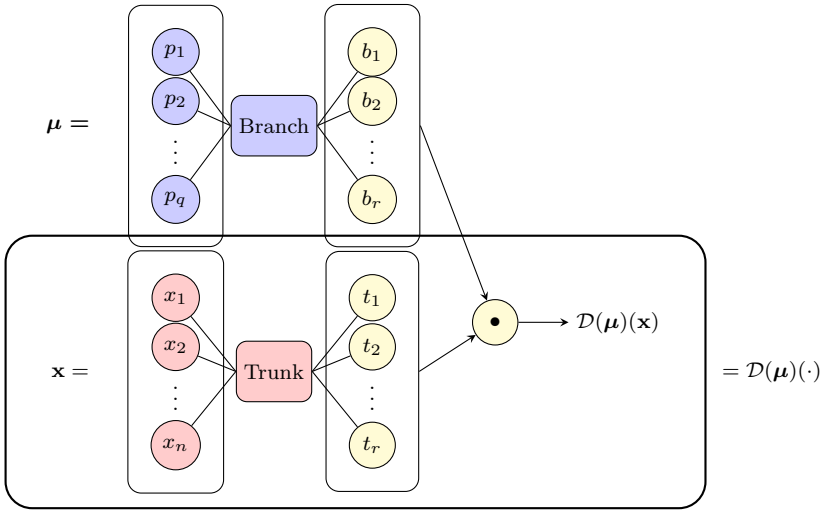


Figure 3.3: DeepONet architecture.

Remark. The `FourierFeatures` layer, which we used only in the trunk network, maps an input $\mathbf{x} \in \mathbb{R}^d$ to a higher-dimensional space using sinusoidal functions. This allows a neural network to capture more easily high-frequency variations in the input. More precisely, given a set of frequencies $\{f_i\}_{i=1}^m$, that in our network are trainable parameters, the layer is defined as:

$$\mathcal{F}(\mathbf{x}) = [\mathbf{x}, \sin(2\pi f_1 \mathbf{x}), \cos(2\pi f_1 \mathbf{x}), \ldots, \sin(2\pi f_m \mathbf{x}), \cos(2\pi f_m \mathbf{x})]\,,$$

where $\sin(2\pi f_i \mathbf{x})$ and $\cos(2\pi f_i \mathbf{x})$ are themselves vectors, obtained just by applying the sine and consine functions component-wise. In the end, the output of the layer has dimension $d + 2md$.

Snapshots grouping and masking. As we said in the preceding paragraph, we don't want to separate data from the same snapshots. Firstly, we put ourselves in the simple case, when the number of points (or mesh elements) doesn't change among the snapshots, i.e. $N_i = N$ for every i. In this situation we may organize our data in three tensors as shown in Table 3.2.

Tensor	Shape	Description
S	$N_s \times N$	snapshot tensor containing solutions
X	$N_s \times N \times d$	coordinates tensor for each snapshot in d dimensions
M	$N_s \times p$	parameter tensor for each snapshot

Table 3.2: Data tensors.

In this way we can easily access all the data related to a specific snapshot i by indexing the first dimension of the tensors. Now, since the network should be able to deal with tensor input, we also have to define again the scalar product between the branch and the trunk networks:

$$\mathcal{D}(M_i)(X_{i,j}) = \mathbf{b}(M_i) \cdot \mathbf{t}(X_{i,j}) = \sum_{k=1}^{r} b_k(M_i)\, t_k(X_{i,j}),$$

where M_i is the i-th row of the parameter tensor M and $X_{i,j}$ is the j-th point of the i-th snapshot in the coordinates tensor X. So the output will be a tensor of shape $N_s \times N$, which is consistent with the shape of the snapshot tensor S. In the implementation, we developed a dedicated layer to handle this kind of data organization, called `EinsumLayer`, which takes as input the tensors S, X and M and outputs the predicted solutions for each snapshot. Now, if we want to consider the more general case where the number of points (or mesh elements) can change among the snapshots, i.e. $N_i \neq N_j$ for some $i \neq j$, we can still use the same organization as before, but with a slight modification. We can define a maximum number of points $N_{max} = \max_{i=1,\ldots,N_s} N_i$ and create tensors of shape $N_s \times N_{max}$ for S and $N_s \times N_{max} \times d$ for X, filling in the extra entries with a special value (e.g. NaN). To handle these extra entries during computations, we can introduce a masking tensor of shape $N_s \times N_{max}$, where each entry is a boolean indicating whether the corresponding entry in S and X is valid (True) or not (False). In this way, when performing operations on the tensors, in particular when computing losses during training, we can simply ignore the entries where the mask is False, ensuring that only valid data points are considered. Operationally, we decided to avoid using a mask, but we opted instead to create a masked loss function that checks if the point is valid looking directly at the snapshot tensor S.

Chapter 4
Repository and Implementation

Overview. The main areas of interest in the repository, which is available at https://github.com/alessandropedone/deeponet-for-mems, are:

- `docs/`: folder containing the Sphinx documentation, which you can also find online at https://alessandropedone.github.io/deeponet-for-mems/;
- `data/`: folder containing the package for data generation and post-processing;
- `surrogate/`: folder containing the code for the surrogate model;
- `multi_physics/`: folder containing the code for the coupled electromechanical solver;
- `test/`: folder containing test cases to validate the code;
- `geometries/`: folder containing the `gmsh` code for the geometries for our test cases;
- `models/`: folder containing the Keras models we trained for our test cases.

Figures 4.1 and 4.2 provides an overview of the repository organization, illustrating the main packages and their relationships through a Mermaid diagram. You can also find:

- *Markdown files* (available in the documentation too): `README.md` containing general information about the repository, which you can take as reference to navigate the repository, `test/run_test_cases.md` containing the instructions to run the test cases present in the `test/` folder and `multi_physics/multi_physics_model.md` which contains the detailed description of how the coupled solver actually works.
- *Executable scripts*: `setup.sh` for setting up quickly the environment, `build_docs.sh` for building again the documentation (when you clone the repository you already find it).

Remark. For the installation `conda` is required.

Data package. The most important module in this section is `data.generate`, which implements the following pipeline:

1. `data.geometry.generate_geometries`, to generate the geometries in `.geo` format;
2. `data.mesh.generate_meshes`, to generate the meshes (`.msh`) from the geometries;
3. `data.dataset.generate_datasets`, to generate the dataset of numerical solutions in `.h5` format through the module `data.fom`.

The resulting output is a folder, chosen by the user, where you can find 3 subfolders (one for geometries, one for meshes, and one for results) and a `.csv` file containing the correspondence between solutions IDs and geometric parameters.

Remark. You can see how to use `data.generate` in the documentation.

The second most important module is `data.plot`, which contains all the functionalities for data visualization. The really interesting function in this module is `data.plot.plot` which changes its behaviour based on the kind of solution (cell-based, vertices-based or `None`) it gets as input. Then, several wrappers of this functions are available for our specific purposes.

Surrogate package. The modules in the package that are supposed to be called by the user are three:

- `surrogate.train`, which is meant for training the network and contains the information of the chosen architecture;
- `surrogate.evaluate`, which allows you to visualize a summary of the network that you are loading, and evaluate it on a test set of your choice;
- `surrogate.predict`, which allows you to visualize graphically, using `data.plot` module, the prediction on a test geometry of your choice.

Remark. As always you can check in the documentation how to use this modules and all the available optional arguments.

While we basically already covered the core usage of the package, there's actually something more, related to the network architecture and the training, which is going on under the hood and may result interesting to some reader who wants to understand, and maybe exploit, this repository. So, we mention the following modules too:

- `surrogate.losses`, which contains the two masked metrics (MSE and MAE) as we discussed when talking about data grouping and masking in Section 3.2;
- `surrogate.model`, which contains the core classes used to build the neural networks and will be discussed in more detail later.

Masking and Mixed-precision. We want to take a closer look at the masked loss. The first lines of the following block of code simply recap what has already been discussed, whereas the last one conceals subtle issues related to floating-point precision. In particular, we can face overflow if the number of points in a batch is very large and `tf.reduce_sum(mask)` exceeds the maximum representable number for the used `dtype`. If you use mixed-precision training, this is a real risk since the default `dtype` for activations is `float16`, which has a maximum representable number of only 65504.0[1].

[1] It's relatively easy to get in batch a number of degrees of freedom which exceeds 65504. For instance, if take a batch of 4 simulation, with $2 \cdot 10^4$ degrees of freedom (mesh vertices for polynomial of degree 1) each, you get a batch of $2 \cdot 10^4 > 65504$.

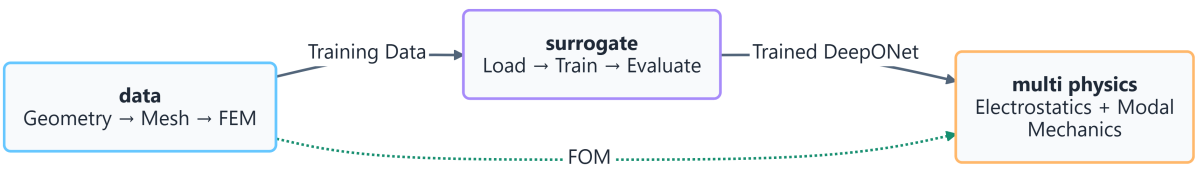


Figure 4.1: Repository packages Mermaid diagram.

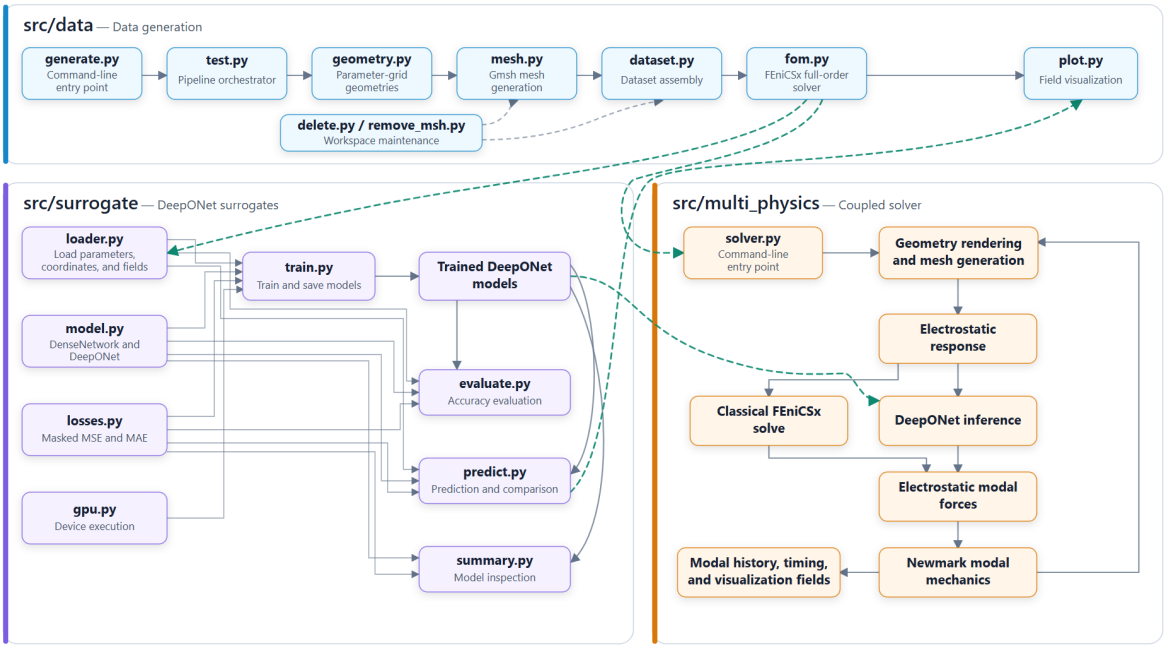


Figure 4.2: Repository packages Mermaid diagram with details.

```python
import tensorflow as tf
from keras.utils import register_keras_serializable
@register_keras_serializable()
def masked_mse(y_true: tf.Tensor, y_pred: tf.Tensor)-> tf.Tensor:
    # Masking from y_true information
    mask = tf.cast(~tf.math.is_nan(y_true), y_pred.dtype)
    # Set to 0 the NaNs values
    y_true = tf.where(tf.math.is_nan(y_true), 0.0, y_true)
    # Make the types coherent
    y_true = tf.cast(y_true, y_pred.dtype)
    # Perform the computation of the masked loss
    return tf.reduce_sum(mask * tf.square(y_pred - y_true))/tf.reduce_sum(mask)
```

To avoid this issue, we decided to use directly `float32` mixed-precision policy for the whole model, which is a reasonable choice since our models are not very large and the memory savings of `float16` are not crucial. In this way, we can be sure that the masked loss function will not overflow during training, since the maximum batch size we can use is well below the `float32` limit of $3.4 \cdot 10^{38}$. However, if one is dealing with big models and really needs to use `float16` for memory savings, a possible solution to avoid overflow in the masked loss function is to cast everything to `float32`, over even `float64`, only inside the loss function, and still keep the rest of the model in `float16`, using the mixed-precision policy setting before training:

```python
from tensorflow.keras import mixed_precision
policy = mixed_precision.Policy('float16')
mixed_precision.set_global_policy(policy)
```

This shows also that our masking technique is the simplest possible implementation and that it's compatible with any use case, unless, of course, you choose to move away from TensorFlow/Keras.

Multi-physics package. The package contains only one module: `multi_physics.solver`. We invite the interested reader to refer to `multi_physics/multi_physics_model.md` and the documentation for a detailed explanation of all the functionalities. Here we recall that the structure have already been discussed here in Section 2.4. There is only one theme which deserves a quick comment, since it represents where the computational gain happens: *post-processing*. At each time step of the simulation of the DL-ROM, you could decide to mesh (very expensive) and visualize the solution or just compute the force without meshing; this allows you to keep the time integration very accurate and *visualize the solution every k time steps*[2]. Moreover, we decided to avoid implementing a module for post-processing, which allows you to visualize the solution over time given the history of modes' coefficients (which is the output for the DL-ROM too) using a potential surrogate. Instead, the frequency (in terms of time steps) of solution post-processing and visualization is a choice you do only at the beginning of each simulation. This works fine since you know a priori the time accuracy you need for both integration and visualization. Even in the case you need a finer accuracy in terms of visualization, it's sufficient to run the simulation from scratch, since in our code the computational cost of visualization is bound to the meshing phase, which is a lot more expensive then the solution itself. In any case, in principle one could decide to implement a personal, and mesh-free, way of visualizing the solution using the potential field surrogate which is already provided in the repository.

[2] You can set this parameter as an optional argument when calling the module.

DeepONet class and Inheritance. Developing this class was an interesting challenge, since Keras models are generally used within wrappers. Initially, we worked with that approach, but when we moved to the DeepONet architecture, the low flexibility of this approach became apparent. The issues with this approach are mainly two:

- the stratified structure of the architecture requires using dense networks as members of the DeepONet;
- the scalar product must allow the network to process tensor inputs.

So, exploiting *inheritance was necessary*. The details can be found in the documentation, but the main challenges of doing this were understanding how to split the various properties of the network between `__init__()`, `call()` and `build()` methods and how to override the configuration saving procedure with the methods `from_config()` and `get_config()`. After solving this problem without extensive and comprehensive documentation, we can say that it's actually pretty simple if you know in advance how your code should be structured.

Tensorflow/Keras vs PyTorch. In academic research, PyTorch is often chosen for its flexibility with runtime changes (like conditional logic inside your network, or, as in our case, variable length sequences), thanks to its dynamic graph computation. But, in the end, TensorFlow/Keras offers remarkable flexibility while providing several *built-in functionalities* that make development smoother. Features like automatic summaries, detailed training history logs with TensorBoard, built-in model checkpointing, early stopping, learning rate scheduling, and high-level APIs such as `model.fit` streamline the entire training workflow. Additionally, Keras handles validation splits, data shuffling, and metric tracking automatically, reducing boilerplate code and making experimentation faster. While PyTorch offers similar capabilities, many of them require manual implementation or external libraries, which makes TensorFlow/Keras particularly appealing for rapid prototyping, especially in applications like ours. As a result, the choice between the two frameworks is not always obvious.

Parallelism. We employed parallel computing techniques, while relying on a Python interface. In particular, there are two areas where this technique significantly improves throughput: the *mesh generation*, which is the real bottleneck of the dataset generation process, and the *training* of the network. Focusing on the data generation section, we decided to implement the same parallel implementation for geometry generation, which is almost useless since the time to modify the text file is really a matter of a few seconds in the worst case, and for mesh generation, where really this idea truly shines. We used the `concurrent.futures` Python module in the following workflow:

1. employ `ProcessPoolExecutor`, a class that runs tasks in separate processes, each with its own Python interpreter to bypass the GIL (Global Interpreter Lock), which is a bottleneck in CPU-heavy tasks;
2. submit independent tasks to the executor using `ProcessPoolExecutor.submit()`, producing `Future` objects;
3. use `as_completed()` to iterate over tasks as they finish, enabling asynchronous result handling;
4. retrieve results with `Future.result()`, blocking only when necessary;
5. each process works on a separate geometry, avoiding race conditions and ensuring deterministic output;
6. control concurrency via the variable `max_workers`, which can be set by user in the `data.generate` call, to balance CPU usage and system resources based on preferences/necessities;
7. track overall progress with `tqdm`.

Regarding the parallelization of the training process, we implemented a wrapper for functions in `surrogate.gpu` that enables the use of TensorFlow on a GPU. This allows any user with a compatible GPU and the required CUDA setup to leverage GPU acceleration during training.

Remark. CUDA requirements depend on your machine, they cannot be generalized and they could be quite tricky to fulfill. Just to give an example, you may need to run:

```
conda activate env_name
conda install cuda-cudart cuda-version=12
```

where `env_name` is the conda environment you built with `setup.sh`.

Flexibility and Extensions. The developed code is designed to be modular, flexible, and easily extensible. In the following quick overview, we highlight possible extensions and adaptations of the three packages.

- The `data` package can be reused for different problem settings. This mainly requires modifying the `data.fom` module to solve the new problem and updating the `.geo` file defining the reference geometry and also the `.csv` file containing the geometric parameters, which is read by `data.generate`.
- In `surrogate` you can exploit as they are the core model architecture implemented in `surrogate.model`, the loss functions defined in `surrogate.losses`, and the GPU function wrapper in `surrogate.gpu`. While the overall structure can be preserved, the methods `surrogate.evaluate`, `surrogate.train`, and `surrogate.predict` must be adapted to handle different datasets.
- The `multi_physics` package is specifically designed for our application and while it is based on the coupling of the classical ROM with the DeepONet, it could be a great idea to substitute it with a purely data-driven surrogate model as well.

Chapter 5
Numerical results

To run these tests yourself, just take a look at the `README.md` file. It redirects you to the `test/run_test_cases.md`, which contains all the instructions.
Remember to run the provided commands from the main directory of the code.

5.1 Introduction

We built and tested three different surrogate models, corresponding to three modeling approaches for the upper plate deformation, which are:

1. *cantilever* with geometric parameters similar to real applications;
2. *cantilever* with more geometric variability, in order to better understand the dependence of problem on the geometry and the approximation capabilities of the surrogate model;
3. *clamped-clamped* with geometric parameters similar to real applications.

Before analyzing their structure and the results we got when coupled with the classical modal projection for the mechanical part, we summarize the geometric variability of our problem. In all three cases we have basically three parameters:

- *distance* between the two plates;
- *over-etch* of the upper plate, which is basically an external volume removed from the ideal geometry parametrized by the width of this over-etch itself;
- *coefficients* of the first 4 modes we kept to approximate the deformation.

Number of modes. Choosing the right number of modes N_m to approximate the displacement heavily depends on the application you are dealing with. In the case of MEMS, empirical evidence suggests us that basically one mode is enough to model the deformation, since typically the deformation oscillates with a frequency, which is low enough with respect to the natural frequencies of the material. However, following this intuition, we kept $N_m = 4$, which should be more than enough, and then verified the validity of our assumption a posteriori for the numerical simulation. Our assumption turned out to be true, as we will see in the section about the Classical ROM.

Outline. We first describe how the `data` and `surrogate` packages are used to train the surrogate models. We then present and validate the multiphysics solver, and finally assess its performance when integrating the pre-trained surrogate models.

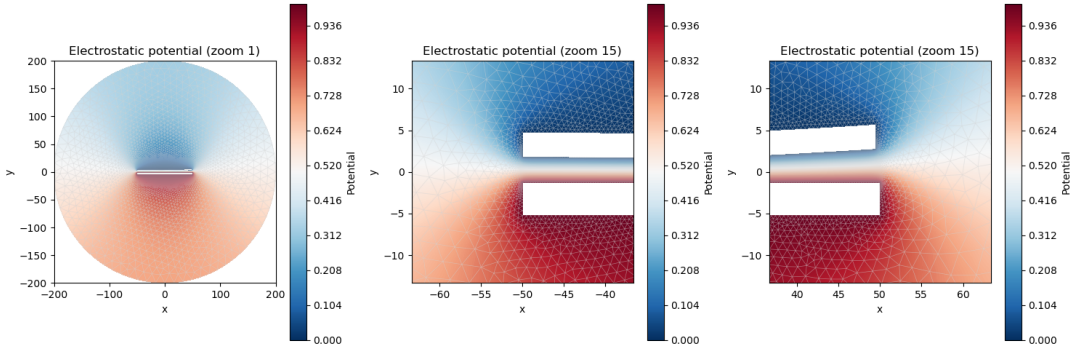
Remark. See the repository for more details on how these results are obtained and can be reproduced.

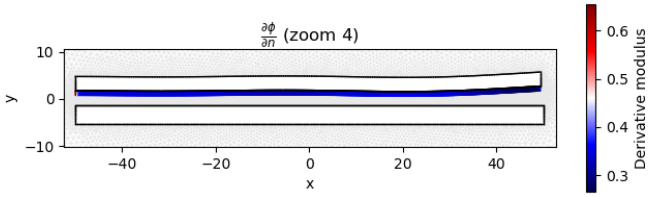
Electrostatics Full Order Model (FOM). In all three cases, we solve problem (1.1) setting $V_l = 1$, $V_u = 0$, and homogeneous Neumann boundary conditions on the outer fictitious boundary. We choose a continuous Lagrange (CG) finite element space of degree 1 for the electrostatic potential ϕ, and for the gradient of the potential, we define a vector-valued discontinuous Galerkin (DG) space of degree 0, which is piecewise constant per element, and captures the exact gradient of the $\mathbb{P}_1$ CG potential, including jumps at element boundaries. Everything is implemented in `data.fom`, using FEniCSx.

5.2 Surrogates

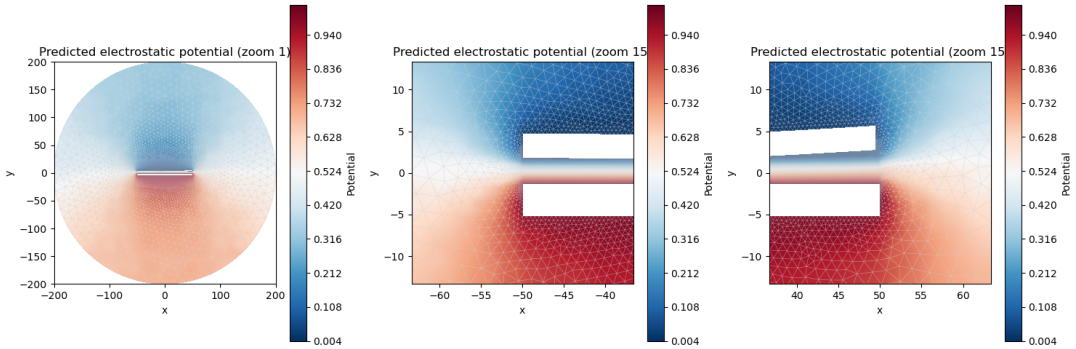
Now we observe the performance of the surrogate model, trained in the three different scenarios. Actually we discuss only the first two, since the third, i.e. the one that deals with the case of a beam clamped at both ends with a physically reasonable deformation, yields very similar results to those obtained in the first case (Test 1).

Test 1. This test deals with the case of a cantilever with physical geometric parameters; the geometric parameters and their respective ranges can be found in Table A.4. First of all, we observe the geometric variability of the solutions with the following plots of the potential and the normal derivative on the upper plate for a random choice of geometric parameters in the ranges.

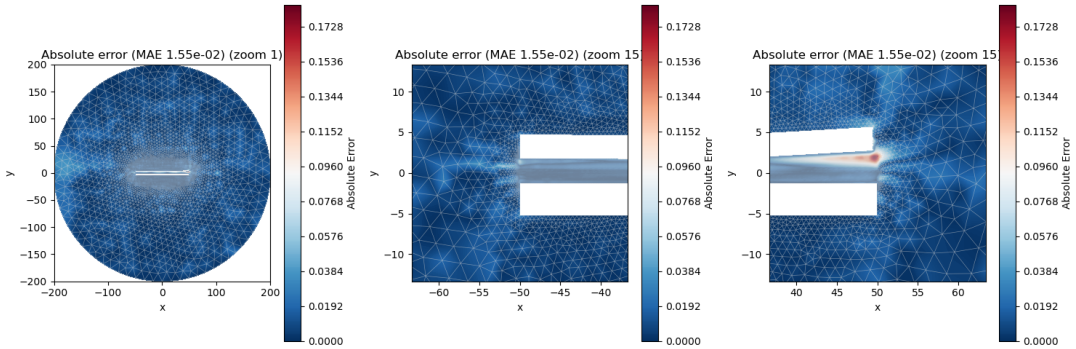




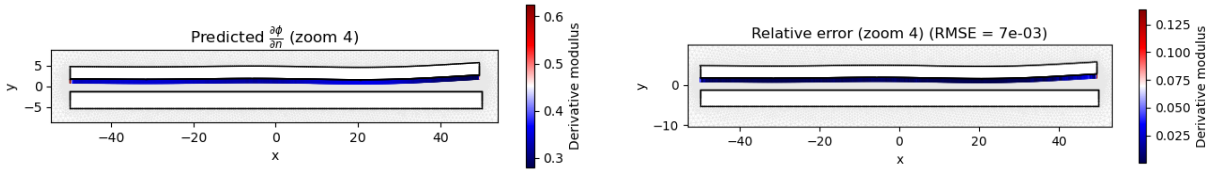
Now we plot the prediction of our model, which is saved in `models/potential1.keras`[1].



And then, we plot the absolute error for potential.



We also take at the normal derivative predicted by `models/derivative1.keras`, with its absolute error.



The relative error is defined empirically as follows:

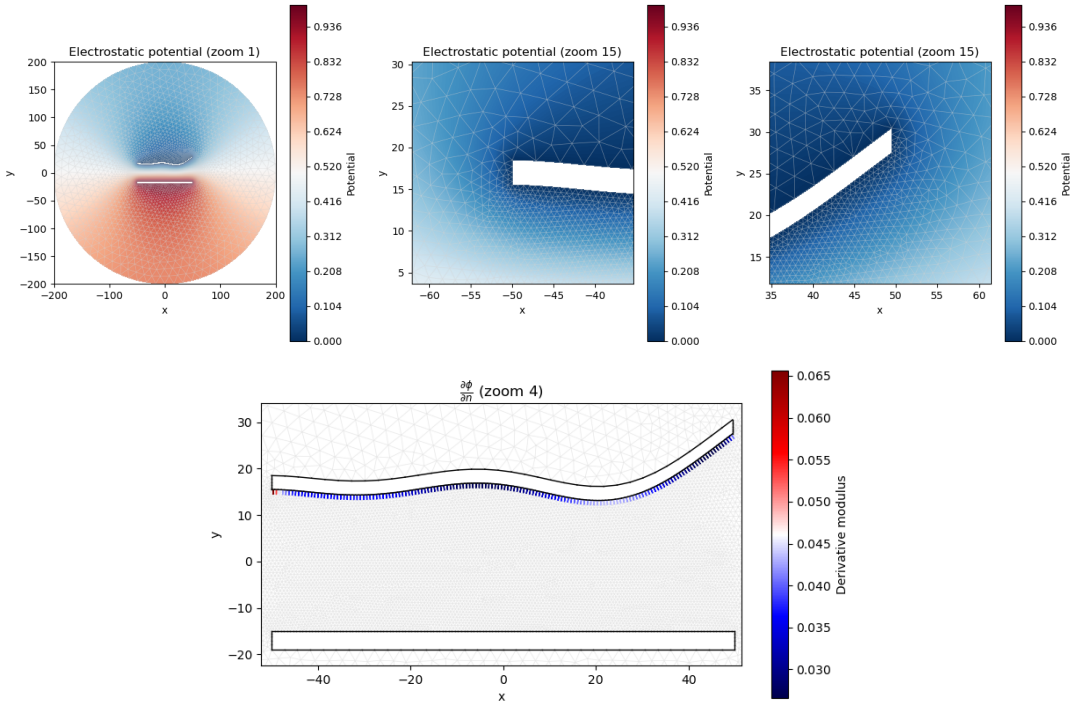
$$\varepsilon_i = \frac{\left|\partial_n \phi_i^{\text{pred}} - \partial_n \phi_i^{\text{ref}}\right|}{\max_j \left|\partial_n \phi_j^{\text{ref}} + \delta\right|},$$

where the indices i and j refer to the cell facet, and $\delta > 0$ is a small constant to avoid dividing by zero. Finally, the performance metrics on the test set are summarized in following table.

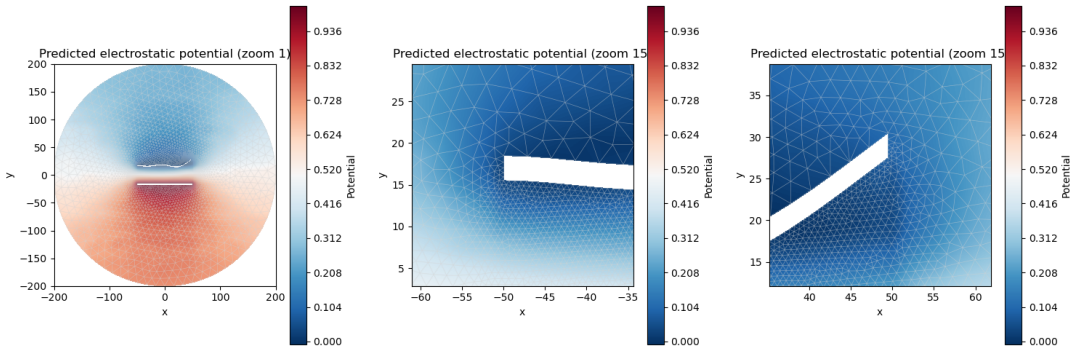
Model	RMSE	MAE
Potential model	≈ 0.0303	≈ 0.0190
Derivative model	≈ 0.0082	≈ 0.0026

Test 2. This test deals with the case of a cantilever with a bigger elastic deformation; the geometric parameters and its respective ranges can be found in Table A.5, the parameters are the same of the previous test but they vary in different ranges.
First of all, with observe the geometric variability of the solutions with the following plots of the potential and the normal derivative on the upper plate for a random choice of geometric parameters in the ranges.

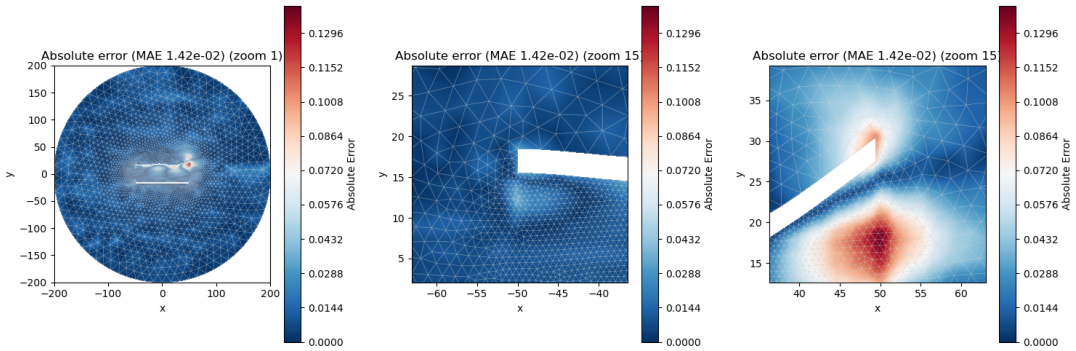
[1] We recall that the surrogates of the electrostatic potential ϕ are intended solely for visualization purposes.



Now we plot the prediction of our model, which is saved in `models/potential2.keras`.



And then, we plot the absolute error for potential.

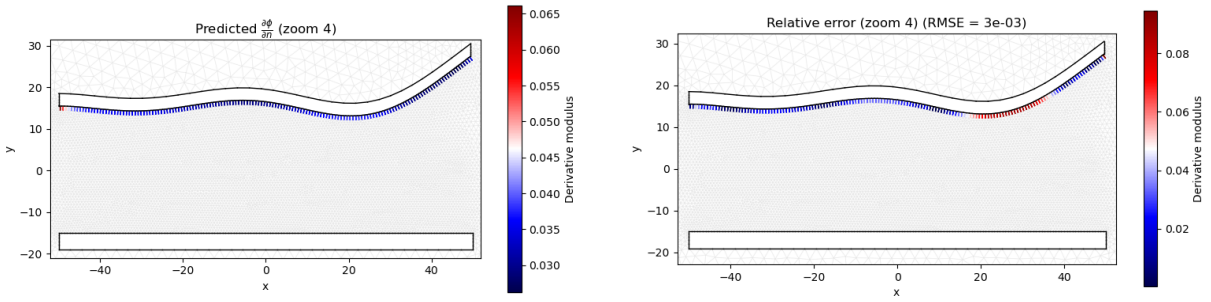


We also take at the normal derivative predicted by `models/derivative2.keras`, with its absolute error.

Remark. Here the relative error seems much less promising than the previous case, but since the distance here is much bigger, we get automatically a smaller gradient of the potential, and indeed the values of the normal derivative on the upper plate are much smaller (see the first of the two plots). So, even if the percentages are bigger, while still being acceptable, the global error is still very good (note that RMSE is of the order of 10^{-3}).

Finally, the performance metrics on the test set are summarized in following table.

Model	RMSE	MAE
Potential model	≈ 0.0295	≈ 0.0185
Derivative model	≈ 0.0028	≈ 0.0020



5.3 Classical Multi-Physics Solver (Classical ROM)

Remark. In the following, for the visualization of time dependent solutions, we suggest to reproduce the results following the instructions provided in the repository. Moreover some videos are already provided as an example. Here we report only static results, like scalar quantities and the capacitance over time, which can be represented by the graph of a function.

Remark. For brevity, from now on we discuss only the situation 1 (i.e. where the cantilever subject to a realistic deformation), since the results are very similar to the other cases.

To validate this solver, we perform two kinds of tests: mesh convergence with respect to the time step and test that number of modes is enough to capture the dynamics of our application. A visual inspection of time dependent solutions provided by this solver is also possible following the instruction in the repository.

Mesh convergence in time. We perform a mesh convergence with this vector of time steps:

$$[10^{-5}, 5 \cdot 10^{-6}, 10^{-6}, 10^{-7}].$$

To compare the different solutions, we take as a reference the grid corresponding to $\Delta t = 10^{-4}$, i.e. a very coarse one. For all choices of the time step, we compute the solution but only save it at the position corresponding to the reference grid. We can exploit this approach to speed up the convergence test: for the intermediate step needed to reach the next reference point, we can use the DL-ROM to integrate the system, assuming that the accuracy is negligible with respect to the error we are trying to assess here. We will verify this hypothesis a posteriori in the next paragraph, and, indeed, it will turn out to be true. Figure 5.1 provides an example of the results of this test, showing the evolution of the capacitance over time for the considered set of time-step sizes in the case of a cantilever (case 1).

Test number of modes. Now we verify that the number of modes $N_m = 4$ is more than enough to solve the problem accurately. In order to do so, we solve the problem using the first mode, then the first two, and so on until we reach all four modes. To conclude this discussion, Figure 5.2 shows the evolution of the capacitance over time for the four different choices of the number of modes in the case of a cantilever (case 1).

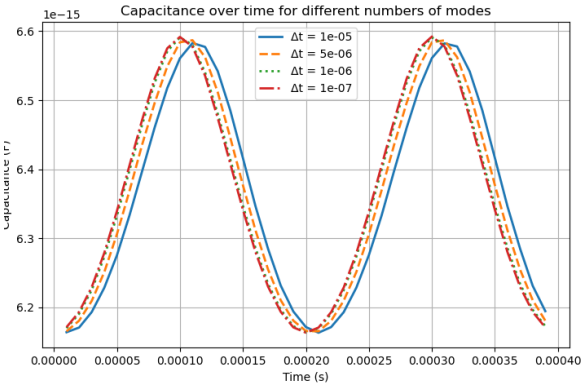


Figure 5.1: Capacitance over time for $\Delta t \in \{10^{-5}, 5 \cdot 10^{-6}, 10^{-6}, 10^{-7}\}$ in the case of a cantilever (1). Mesh convergence is reached for $\Delta t = 10^{-6}$.

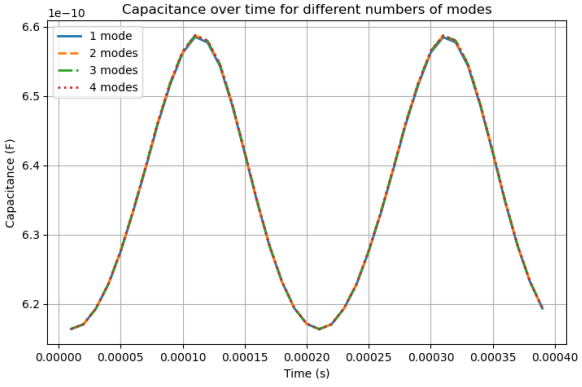


Figure 5.2: Capacitance over time when considering the first 1, 2, 3, and 4 modes in the case of a cantilever (1). As expected, the only relevant mode is the first one.

5.4 DL Multi-Physics Solver (DL-ROM)

In this paragraph we evaluate the performance of the DL-ROM. First, for a specific choice of geometric parameters (H, O) and then for a grid of values.

Performance. First, we choose $(H, O) = (1.5\mu m, 0\mu m)$, and then we run the same simulation as before, using $\Delta t = 10^{-5}$[2]. To investigate the results, we use both qualitative and quantitative approaches and divide the discussion between accuracy and execution time.

[2]We choose a coarse grid since we are not interested in the general quality of the approximation itself. We are only comparing the two models and, hopefully, they should behave similarly for any choice of the time step.

Accuracy. First, we analyze the accuracy of the DL-ROM by considering two error metrics:

- normalized capacitance error

$$\varepsilon_C = \frac{\|C_{\text{pred}} - C_{\text{ref}}\|_{L_t^\infty}}{\max C_{\text{ref}} - \min C_{\text{ref}}},$$

  where C_{ref} denotes the reference capacitance obtained with the Classical ROM and C_{pred} the capacitance predicted by the DL-ROM.
- normalized displacement error

$$\varepsilon_u = \frac{\|u_{\text{pred}} - u_{\text{ref}}\|_{L_x^\infty L_t^\infty}}{\max u_{\text{ref}} - \min u_{\text{ref}}},$$

  where u_{ref} denotes the reference displacement obtained with the Classical ROM and u_{pred} the displacement predicted by the DL-ROM.

For a cantilever (case 1), the errors are

$$\varepsilon_C \approx 0.02195, \qquad \varepsilon_u \approx 0.01042.$$

These values indicate satisfactory agreement between the DL-ROM and the Classical ROM. This is also consistent with the qualitative comparison shown in Figure 5.3, where the capacitance evolution predicted by the two models is depicted.

Remark. The error here is much smaller than the error we wanted to control during mesh convergence. This confirms the validity of the hypothesis introduced previously.

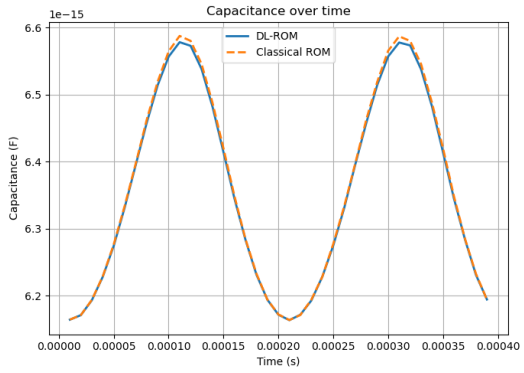


Figure 5.3: Classical ROM and DL-ROM predictions of the capacitance over time with $\Delta t = 10^{-5}$ for cantilever case (1).

Execution time. We now compare the execution times of the DL-ROM and the Classical ROM. The results are summarized in Table 5.1.

Metric	DL-ROM	Classical ROM
Total runtime	3.31 s	57.41 s
Postprocessing/meshing time	1.57 s	56.80 s
Solution time	1.74 s	0.61 s
Overall speedup	**17.34×**	

Table 5.1: Computational performance comparison between the DL-ROM and the Classical ROM.

As expected, the largest computational gain is observed in the postprocessing/meshing stage, since conceptually we are just skipping the meshing phase. A small time dedicated to meshing is still present because the first time step is always solved using the mesh, in order to estimate the fringing effects and translate properly the profile of the capacitance. The solution itself is faster for the Classical ROM, since the FEniCSx solution of the laplacian is very well optimized, while our implementation in Python is clearly slower. In any case, this is not the real bottleneck of this problem.
At this point, the main computational advantage of the DL-ROM becomes clear. The model is particularly beneficial when a very fine time step is required to ensure accurate time integration, while only a limited number of frames need to be visualized (by meshing), e.g. one frame every ten time steps. In this setting, the DL-ROM can significantly reduce the overall computational cost by avoiding expensive meshing operations at every time step.

Remark. Obviously the computational gain depends on the machine you are working on and how much expensive the meshing phase (which is directly influenced by the geometry and mesh size).

Geometry parameters. In this paragraph we ask the question: did we just spot the lucky case? The answer is no, since testing a 3×3[3] grid of values of (H, O) in Table A.4 and gathering the results, yields the metrics in Table 5.2.

Metric	Mean	Std. dev.
Speedup	19.48×	4.465×
ε_C	0.01572	0.006934
ε_u	0.01822	0.009661

Table 5.2: Results of the evaluation of the performance of the model over a 3×3 grid of values of (H, O).

Memory profiling. To perform memory profiling we use Memray. In Figure 5.4 we plot the memory evolution over time for the case 1, with the same specific choice of (H, O) we did in the accuracy test.

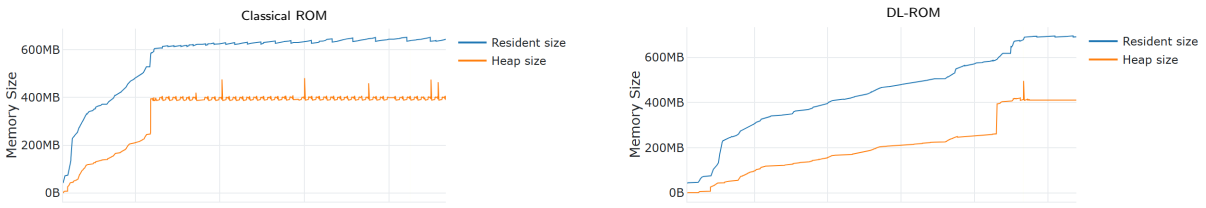


Figure 5.4: Memory usage over time for the Classical ROM and DL-ROM for test 1. Note: the two plots use different time scales! The execution of DL-ROM requires a bit more memory (the peak is higher), due to the presence of a neural network.

5.5 Conclusions

From the results of the three test cases, we can conclude that the DeepONet surrogate model effectively approximates the electric potential and its normal derivative on the upper plate across varying geometries, that the classical ROM has been correctly validated and yields physically sound solutions, and finally that the performance of the DL-ROM can be considered satisfactory in terms of both computational efficiency, as reflected by the achieved speedup, and accuracy.

[3] This is an hyperparameter and can be increased.

Appendix A
Tables

A.1 DeepONet

Table A.1: DeepONet Model Suμmary

Layer (type)	**Output Shape**	**Param #**
DenseNetwork (Branch)	20	19,905
DenseNetwork (Trunk)	20	24,515
Total params		130,410
Trainable params		42,994
Non-trainable params		1,426
Optimizer params		85,990

Table A.2: Branch Network Suμmary

Layer (type)	**Output Shape**	**Param #**
Normalization	6	13
Dense	128	896
BatchNormalization	128	512
Dropout	128	0
Dense	64	8,256
BatchNormalization	64	256
Dropout	64	0
LayerNormalization	64	128
Dense	32	2,080
BatchNormalization	32	128
Dropout	32	0
LayerNormalization	32	64
Dense	128	4,224
BatchNormalization	128	512
Dropout	128	0
LayerNormalization	128	256
Dense	20	2,580
Total params		19,905
Trainable params		19,188
Non-trainable params		717

Table A.3: Trunk Network Suμmary

Layer (type)	**Output Shape**	**Param #**
Normalization	2	5
FourierFeatures	42	10
Dense	128	5,504
BatchNormalization	128	512
Dropout	128	0
Dense	64	8,256
BatchNormalization	64	256
Dropout	64	0
LayerNormalization	64	128
Dense	32	2,080
BatchNormalization	32	128
Dropout	32	0
LayerNormalization	32	64
Dense	128	4,224
BatchNormalization	128	512
Dropout	128	0
LayerNormalization	128	256
Dense	20	2,580
Total params		24,515
Trainable params		23,806
Non-trainable params		709

A.2 Parameters

Parameter	Min	Max	Steps
Over-etch (μm)	0.0	0.5	5
Distance (μm)	1.5	2.5	5
C_1	-0.12	0.12	3
C_2	-0.12	0.12	3
C_3	-0.12	0.12	3
C_4	-0.12	0.12	3

Table A.4: Geometric variability in Test 1.

Parameter	Min	Max	Steps
Over-etch (μm)	0.0	0.5	5
Distance (μm)	1.5	2.5	5
C_1	-1.5	1.5	3
C_2	-1.5	1.5	3
C_3	-1.5	1.5	3
C_4	-1.5	1.5	3

Table A.5: Geometric variability in Test 2.

Parameter	Min	Max	Steps
Over-etch (μm)	0.0	0.5	5
Distance (μm)	1.5	2.5	5
C_1	-0.12	0.12	3
C_2	-0.12	0.12	3
C_3	-0.12	0.12	3
C_4	-0.12	0.12	3

Table A.6: Geometric variability in Test 3.

Bibliography

[BPS06] Romesh C. Batra, Maurizio Porfiri, and Davide Spinello. Electromechanical model of electrically actuated narrow microbeams. *Journal of Microelectromechanical Systems*, 15(5):1175–1189, 2006.

[EBw25] Euler–Bernoulli beam theory. Wikipedia Link, 2025.

[FDM21] Stefania Fresca, Luca Dedè, and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized PDEs. *Journal of Scientific Computing*, 87(2):61, 2021.

[HKA+24] Junyan He, Seid Koric, Diab Abueidda, Ali Najafi, and Iwona Jasiuk. Geom-deeponet: A point-cloud-based deep operator network for field predictions on 3d parameterized geometries. *Computer Methods in Applied Mechanics and Engineering*, 429:117130, September 2024.

[HLK00] Yoon Shik Hong, Jong Hyun Lee, and Soo-Hyun Kim. Laterally driven symmetric micro-resonator for gyroscopic applications. *Journal of Micromechanics and Microengineering*, 10(3):452, 2000.

[KS19] Firas A. Khasawneh and Daniel J. Segalman. Exact and numerically stable expressions for Euler–Bernoulli and Timoshenko beam modes. *Applied Acoustics*, 151:215–228, 2019.

[LJK19] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *CoRR*, abs/1910.03193, 2019.

[LJP+21] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, Mar 2021.

[LW16] Philippe Laurençot and Christoph Walker. Some singular equations modeling mems. *Bulletin of the American Mathematical Society*, 54(3):437–479, December 2016.

[MAM18] Ramin Mirzazadeh, Saeed Eftekhar Azam, and Stefano Mariani. Mechanical characterization of polysilicon mems: A hybrid tmcmc/pod-kriging approach. *Sensors*, 18(4):1243, 2018.

[New59] Nathan M. Newmark. A method of computation for structural dynamics. *Journal of the Engineering Mechanics Division*, 85(3):67–94, 1959.

[You11] Mohammad I. Younis. *MEMS Linear and Nonlinear Statics and Dynamics*, volume 20 of *Microsystems*. Springer, New York, 2011.

[ZRG+24] Filippo Zacchei, Francesco Rizzini, Gabriele Gattere, Attilio Frangi, and Andrea Manzoni. Neural networks based surrogate modeling for efficient uncertainty quantification and calibration of mems accelerometers. *International Journal of Non-Linear Mechanics*, 167:104902, 2024.

[Öc21] Andreas Öchsner. *Classical Beam Theories of Structural Mechanics*. Springer Cham, 2021.